

What is Tokenization?

Tokenization is the process of converting data (email, user id, etc.) into a secure signed string called a token.

The token can safely be sent in URLs, emails, or forms.

Later the server can verify and decode the token using the secret key.

Example:

Original Data → abc@gmail.com

Token → ImFiY0BnbWFpbC5jb20i.Ze1F4g.QJ1g23sdFsd

Example token:

ImFiY0BnbWFpbC5jb20i.Ze1F4g.QJ1g23sdFsd

Inside the token:

email + timestamp + signature

Example token:

ImFiY0BnbWFpbC5jb20i.Ze1F4g.QJ1g23sdFsd

It looks random, but it has 3 parts.

♦ Inside the Token (3 Parts)

Think of the token like a **sealed cover** 

1 First Part → DATA (Payload)

ImFiY0BnbWFpbC5jb20i

👉 This contains your actual data (like email)

- Usually encoded (Base64)
- Example inside:

abc@gmail.com

👉 So this part = **What you want to store**

2 Second Part → TIMESTAMP

Ze1F4g

👉 This tells **when the token was created**

- Used to check expiry
- Example:
 - Token created at 10:00 AM
 - Expire after 10 minutes

👉 So this part = **When it was created**

3 Third Part → SIGNATURE

QJ1g23sdFsd

👉 This is the **security part**

- Created using a secret key
- Ensures:
 - Data is not changed
 - Token is valid

👉 If someone changes email → signature becomes invalid ❌

👉 So this part = **Security verification**

♦ Simple Formula

TOKEN = DATA + TIME + SIGNATURE

♦ Why do we use tokens?

Instead of storing data directly or exposing it, we use tokens to make it:

- Secure 🔒
- Tamper-proof 🚫
- Time-limited ⌚

What is itsdangerous?

itsdangerous is a Python library used in Flask to generate and verify secure tokens.

Common uses:

- Session cookies
- CSRF tokens
- Email verification links
- Password reset links
- OTP verification

URLSafeTimedSerializer

It creates tokens that:

- are safe to use in URLs
- include timestamp
- support token expiration

Example:

```
from itsdangerous import URLSafeTimedSerializer
```

```
serializer= URLSafeTimedSerializer(secret_key="2345654",salt="Email_verify")
```

`dumps()`

Used to generate token

```
token = serializer.dumps(email)
```

loads()

Used to verify token and retrieve original data

```
email = serializer.loads(token, max_age=300)
```

If token expired → SignatureExpired

If token modified → BadSignature

Install it:

```
pip install itsdangerous
```

✓ URLSafeTimedSerializer

It is a class from itsdangerous which helps to:

- Generate tokens that are safe to send in URLs
- Set expiry time for the token (important for OTP)

Example:

```
from itsdangerous import URLSafeTimedSerializer
```

✓ Constructor Parameters

```
serializer = URLSafeTimedSerializer(secret_key, salt='email-confirm')
```

Parameter Meaning

secret_key A private key of your app used for security. Never share it.

salt Extra security layer. You can use different salts for different use cases.

✓ Token Encoding Function — dumps()

dumps() converts any data into a token.

```
token = serializer.dumps(email)
```

✓ Token Decoding Function — loads()

loads() converts token back to original data and also checks expiry.

```
email = serializer.loads(token, max_age=300) # 300 seconds = 5 minutes
```

If token is expired → It will throw an error.

Flow:

User enters email



Server generates token



Send token to user



User returns token (user enters token)



Server verifies token

♦ What is this token?

In Flask (especially when using **itsdangerous**), a token is a **secure string** that stores some data (like email) in an encoded and signed format.

♦ Real-Life Use Cases

✅ 1. Password Reset Link

When user clicks "Forgot Password":

- You generate token with email
- Send link:

/reset-password/<token>

- When user clicks:
 - Decode token
 - Check expiry
 - Allow reset

✓ 2. Email Verification

- User registers
- Send verification link with token
- Token contains email
- When clicked → verify user

✓ 3. Login Sessions (Lightweight)

- Store user info in token instead of DB session
- Useful in APIs

✓ 4. Secure Data Sharing

- Pass data between pages securely without exposing it
-
-

🔥 OTP Verification Example in Flask

Step 1: Install requirements

```
pip install flask itsdangerous
```

Step 2: Code

```
from flask import Flask, request, render_template_string
```

```
from itsdangerous import URLSafeTimedSerializer, SignatureExpired, BadSignature
```

```
app = Flask(__name__)
```

```
secret_key = "mysecretkey123"
```

```
serializer = URLSafeTimedSerializer(secret_key, salt="otp")
```

```
@app.route("/")
def home():
    return '''
    <form action="/send_otp" method="POST">
        Enter Email: <input type="email" name="email">
        <input type="submit" value="Send OTP">
    </form>
    '''
```

```
@app.route("/send_otp", methods=["POST"])
def send_otp():
    email = request.form['email']
    token = serializer.dumps(email) # Generate token

    # Normally we send token via email. But here we show directly:
    return f"Your OTP Token: <b>{token}</b><br><br>" \
        f"<form action='/verify' method='POST'>" \
        f"<input type='hidden' name='token' value='{token}'>" \
        f"Enter OTP Token here:<input name='token_entered'>" \
        f"<input type='submit'></form>"
```

```
@app.route("/verify", methods=["POST"])
def verify():
    token_user_entered = request.form['token_entered']
    try:
        email = serializer.loads(token_user_entered, max_age=120) # 120 sec expiry
```

```
return f"✅ OTP Verified Successfully for email: {email}"
```

```
except SignatureExpired:
```

```
    return "❌ OTP Expired"
```

```
except BadSignature:
```

```
    return "❌ Invalid OTP Token"
```

```
app.run(debug=True)
```

🔍 What Happened Here?

Step	Explanation
User enters email	We generate token using dumps()
Token displayed	(In real project you will send it via email/SMS)
User enters token again	We verify token using loads()
Token expired?	It shows OTP Expired
Token is fake?	It shows Invalid OTP Token
Token valid?	OTP verified ✅

🎯 Summary (Easy to Remember)

Term	Meaning
Tokenization	Convert data → Secure token
itsdangerous	Library for secure tokens
URLSafeTimedSerializer	Creates tokens that expire
secret_key	Main security key
salt	Extra security layer
dumps()	Encode → Create token
loads()	Decode → Get data (check expiry)

A perfect real-time example of tokenization in Flask is **Email Verification after User Registration**.

Almost every website does this.

Example companies:

- Google
- Amazon

When you sign up, they send a **verification link** like:

`https://website.com/verify?token=abc123xyz`

That **token contains your email securely signed**.

Real-Time Example: Email Verification using Token (Flask)

Step 1 — Install packages

```
pip install flask itsdangerous
```

Step 2 — Complete Example Code

```
from flask import Flask, request
from itsdangerous import URLSafeTimedSerializer, SignatureExpired, BadSignature
```

```
app = Flask(__name__)
```

```
# Secret key for signing token
SECRET_KEY = "supersecretkey"
```

```
# Create serializer
serializer = URLSafeTimedSerializer(SECRET_KEY, salt="email-confirm")
```

```
# -----
# Registration Page
# -----
@app.route("/")
def register():
    return ""
```

```

<h2>User Registration</h2>
<form action="/send_verification" method="POST">
    Enter Email: <input type="email" name="email">
    <input type="submit" value="Register">
</form>
'''

```

```

# -----
# Generate Token and Send Link
# -----
@app.route("/send_verification", methods=["POST"])
def send_verification():
    email = request.form["email"]

    # Create token
    token = serializer.dumps(email)

    # Normally this link is sent via email
    verification_link = f"http://127.0.0.1:5000/verify/{token}"

    return f"""
    <h3>Registration Successful!</h3>
    Click the verification link:<br><br>
    <a href="{verification_link}">{verification_link}</a>
    """

```

```

# -----
# Verify Token
# -----
@app.route("/verify/<token>")
def verify(token):

    try:
        email = serializer.loads(token, max_age=300) # 5 minutes expiry
        return f"✅ Email verified successfully for {email}"

    except SignatureExpired:
        return "❌ Verification link expired"

```

```
except BadSignature:  
    return "❌ Invalid verification link"
```

```
app.run(debug=True)
```

How This Works (Simple Flow)

```
graph TD  
    A[User registers] --> B[Server creates token using email]  
    B --> C[Server sends verification link]  
    C --> D[User clicks the link]  
    D --> E[Server verifies token]  
    E --> F[Email verified]
```

Important Line

Generate token:

```
token = serializer.dumps(email)
```

Verify token:

```
email = serializer.loads(token, max_age=300)
```

```
max_age=300
```

Token expires after 5 minutes

Why This Is Called Tokenization

Because this happens:

Original data → abc@gmail.com

Converted into:

Secure token → ImFiY0BnbWFpbnC5jb20i.ZfAb12.QK9dk39sdf

Later decoded back to:

abc@gmail.com

Why Real Companies Use This

Advantages:

- ✓ No database required
- ✓ Secure (signed with secret key)
- ✓ Expiry built-in
- ✓ Easy verification

Perfect for:

- Email verification
 - Password reset links
 - Login links
-

Tokenization is the process of converting data into a **secure signed token** so it can be safely transmitted and later verified without storing it in the database.

Now we will generate a 6-digit OTP, send it to the user's email, and then verify it.

We will not use token for verification now.

We will store OTP temporarily (in a dictionary / session) and compare it.

Step 1: Install Flask

pip install flask

Step 2: Get Gmail App Password (Important)

Follow these steps:

1. Go to <https://myaccount.google.com/>
2. Enable 2-Step Verification
3. Search "App Passwords"
4. Create → Select Mail → Select Your Device
5. Copy the 16-character password (e.g., abcd efgh ijkl mnop)

This will be used as your email sending password.

Full Flask Code – Send and Verify Numeric OTP

Import necessary modules

from flask import Flask, request # Flask for web app, request to access form data

import random # To generate random OTP

import smtplib # To send emails

from email.mime.text import MIMEText # To format email content

Create Flask application

```
app = Flask(__name__)

# Temporary storage for OTPs (Dictionary: key=email, value=otp)
otp_storage = {}

# Email configuration (Use your Gmail and App Password here)
SENDER_EMAIL = "yourgmail@gmail.com"      # Your Gmail ID
SENDER_PASSWORD = "your_16_digit_app_password" # App Password (not normal Gmail password)

# Function to send email
def send_email(receiver, otp):
    # Email subject and body message
    subject = "Your OTP Verification Code"
    body = f"Your OTP Code is: {otp}\n\nThis code is valid for 2 minutes."

    # Create the email structure
    msg = MIMEText(body)    # Set email body
    msg["Subject"] = subject # Set subject
    msg["From"] = SENDER_EMAIL # Sender email
    msg["To"] = receiver    # Receiver email

    # Connect to Gmail SMTP server and send email
    with smtplib.SMTP("smtp.gmail.com", 587) as server:
        server.starttls()          # Start secure connection
        server.login(SENDER_EMAIL, SENDER_PASSWORD) # Login using sender credentials
        server.send_message(msg)   # Send the email message
```

```
# Route for home page (User enters email)

@app.route("/")
def home():
    return '''

    <form action="/send_otp" method="POST">

        Enter Email: <input type="email" name="email" required>

        <input type="submit" value="Send OTP">

    </form>

    '''

# This creates a simple HTML form:

# User enters email → Form sends POST request to /send_otp.
```

```
# Route to generate and send OTP

@app.route("/send_otp", methods=["POST"])
def send_otp():
    email = request.form['email'] # Get email entered by user from form

    # Generate a 6-digit random OTP
    otp = random.randint(100000, 999999)

    # Store this OTP in dictionary with email as key
    otp_storage[email] = otp

    # Send OTP to the user's email
    send_email(email, otp)

    # Return HTML for OTP input page
    return f'''
```

✅ OTP has been sent to your Email: {email}


```
<form action="/verify" method="POST">
```

```
    <input type="hidden" name="email" value="{email}">
```

```
    Enter OTP: <input type="text" name="otp_entered">
```

```
    <input type="submit" value="Verify OTP">
```

```
</form>
```

```
'''
```

```
# This page asks user to enter OTP for verification
```

```
# Route to verify OTP
```

```
@app.route("/verify", methods=["POST"])
```

```
def verify():
```

```
    email = request.form['email']          # Get user's email again from hidden input
```

```
    otp_entered = request.form['otp_entered'] # Get OTP entered by user
```

```
# Check if:
```

```
# 1. Email is present in OTP storage
```

```
# 2. Entered OTP matches stored OTP
```

```
if email in otp_storage and str(otp_storage[email]) == otp_entered:
```

```
    return f"✅ OTP Verified Successfully for Email: {email}"
```

```
else:
```

```
    return "❌ Invalid OTP! Please Try Again."
```

```
# Run the Flask app
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

🧠 How This Works?

Step	What Happens
User enters email	We generate 6-digit OTP
OTP stored in otp_storage dictionary	For temporary verification
OTP mailed to user	Using Gmail SMTP
User enters OTP	We compare stored OTP vs entered OTP
Match	✅ Verification successful
Not Match	❌ Invalid OTP

🎯 Key Points to Remember

Concept	Meaning
otp_storage	Temporary memory to hold OTPs
random.randint()	Generates random 6-digit OTP
request.form[]	To get data entered in form
send_email()	Sends OTP to user's email
SMTP server (smtp.gmail.com)	Used to send email through Gmail
App Password	Required instead of Gmail login password